Polytime Reductions: A Recap

Recall from last time:

Definition: We say A is polytime reducible to B ($A \leqslant_p$) iff there is a polytime computable function $f : \Sigma^* \to \Sigma^*$ such that

$$\forall w \in \Sigma^*, w \in A \leftrightarrow f(w) \in B.$$

Theorem 7.31:

If $A \leqslant_p B$ and $B \in \mathrm{P}$, then so is A.

If $A \leqslant_p B$ and $B \in \mathrm{NP}$, then so is A.

Corollary:

If $A \leqslant_p B$ and $A \notin \mathrm{P}$, then neither is B.

If $A \leqslant_p B$ and $A \notin \mathrm{NP}$, then neither is B.

If $A \leqslant_m B$ and A is not decidable, then neither is B.

If $A \leqslant_p B$ and A cannot be solved in polytime, then neither can B.

Hardness and Completeness: A Recap

Definition: For any class $\mathcal{C}$ of languages, we say that a language L is *hard* for that class if every $A \in \mathcal{C}$ reduces to it.

- *So $HALT$ is hard for the recognizable languages.*

Definition: For any class $\mathcal{C}$ of languages, we say that a language L is *complete* for that class if $L \in \mathcal{C}$ and every $A \in \mathcal{C}$ reduces to it.

- *So $HALT$ is complete for the recognizable languages.*

If a language L is $\mathcal{NP}$-complete, then:

- If $L \in \mathcal{P}$, then $\mathcal{P} = \mathcal{NP}$.
- Since we strongly believe that $\mathcal{P} \neq \mathcal{NP}$, we strongly believe that $L \notin \mathcal{P}$.

The Cook-Levin Theorem: The language 3SAT is $\mathcal{NP}$-complete.

Arguing that an L is (probably) not in $\mathcal{P}$

We haven't yet proven the $\mathcal{P} \neq \mathcal{NP}$ conjecture, but we can give strong evidence that language is not in $\mathcal{P}$: we can show that it is $\mathcal{NP}$-hard.

To show an L is $\mathcal{NP}$-hard, we reduce from a known $\mathcal{NP}$-complete problem like 3SAT.

So what does a polytime reduction look like?

This is the bad news.

> We won't be using a single format of proof this time in the way that we did when proving undecidability. Since the problems we will be looking at are more varied (e.g., embedding questions about boolean formulas into questions about cliques in graphs), no single template will work quite as well.
>
> *We'll have to vary how we approach these reductions in order to fit the different languages we see. We'll try to cover several languages so you can have examples to learn from, but there are a number of useful ways to start our reductions.*

Example 1: 3COL

Let's show (assuming 3SAT is $\mathcal{NP}$-complete) that 3COL is $\mathcal{NP}$-complete.

I.e., we'll show that $\text{3SAT} \leqslant_p \text{3COL}$.

. . .**OK, but how can we do this!?**

Well, we're doing polytime reduction – that's a type of mapping reduction. So:

- We take as input a 3CNF $\langle \phi \rangle$.
- We return a graph $\langle G \rangle$.
- G should be 3-colourable iff ϕ is satisfiable.
- The whole construction takes a polynomial amount of time.

Important: We don't think we can tell in polytime whether ϕ is satisfiable. So we can't solve ϕ as part of the reduction.

Example 1: 3COL (2)

What we do instead: We'll pass off the task of determining whether $\phi \in$ 3SAT to whoever tries to determine if $G \in$ 3COL.

- We will write a graph G.
- *Someone else* will try to 3-colour it.
- As we write G we'll try to constrain the colour choices available to the searcher.

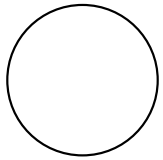
  The choices will reflect the choices we'd make in a search for a satifying truth assignment for ϕ.

. . .***OK, that's start, but how?***

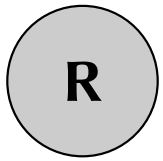
If we don't know how to program 3SAT into 3COL, let's start playing with different 3COL instances to see how to program *anything* into it. We'll build up from there.

3COL: A Start

So let's start with a very simple graph: just the one node.



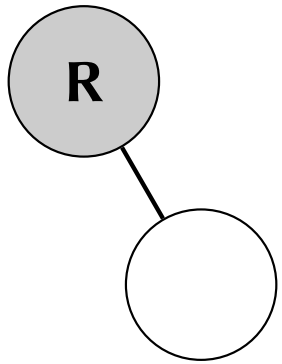
This is easy for our searcher to colour:



The colour **R** *(red) is arbitrary. It could be green or blue, as well.*

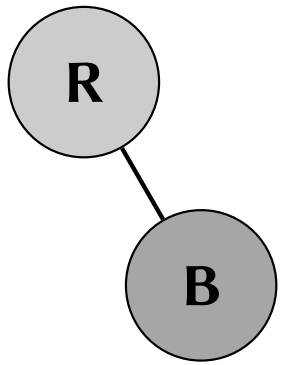
3COL: A Start (2)

Let's make things bit harder, then:



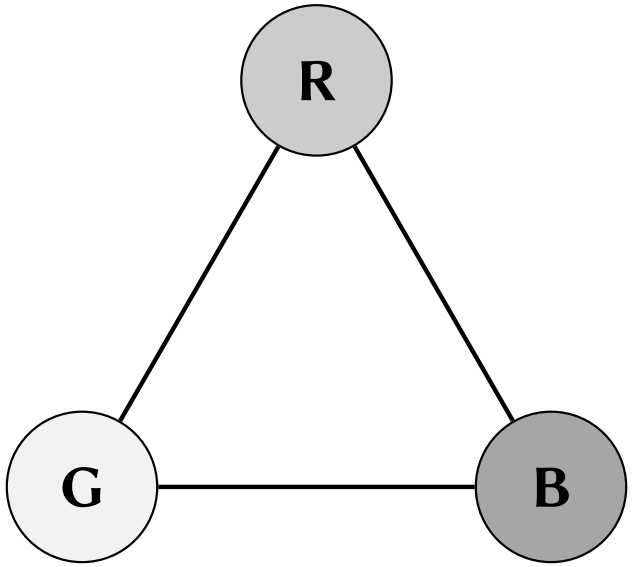
We're looking at how to extend the colouring we've got.

There are two possibilities for the new node – let's colour it blue.



3COL: A Start (3)

Let's add one more node:

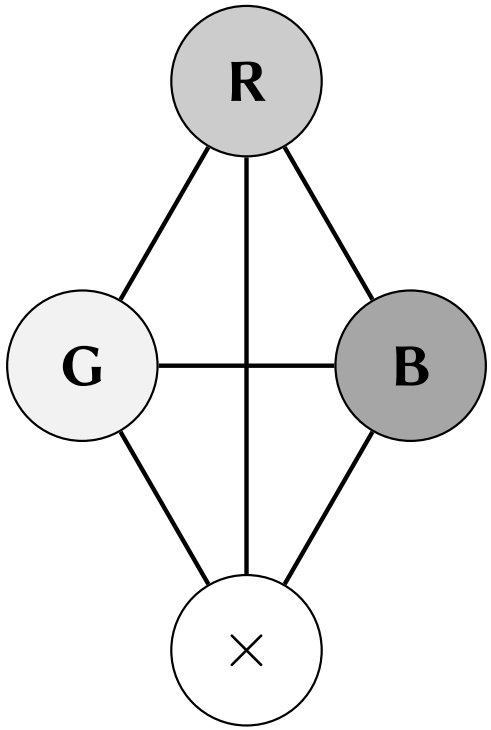


This time there's only one choice of colour.

- The first two choices were arbitrary, but now the choice is forced.
- *Remember that we're trying to force our searcher to make certain colour choices. This is the behaviour we're looking for.*

3COL: A Start (4)

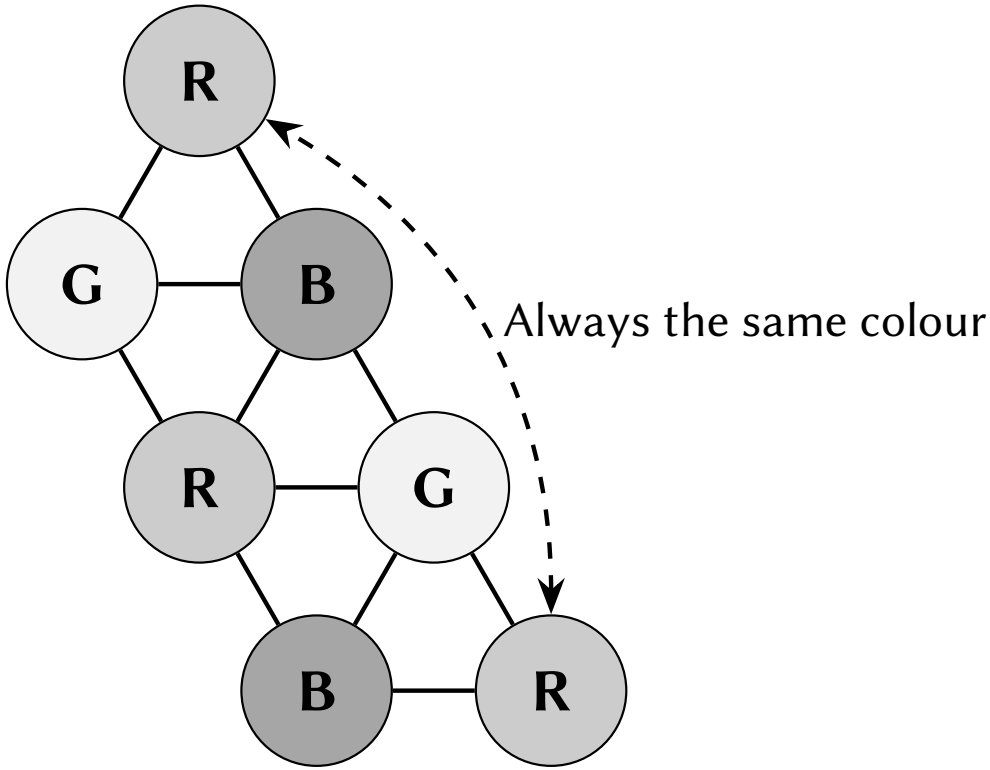
We can't add more totally connected nodes:



So our triangle is maximally constrained.

3COL: A Start (5)

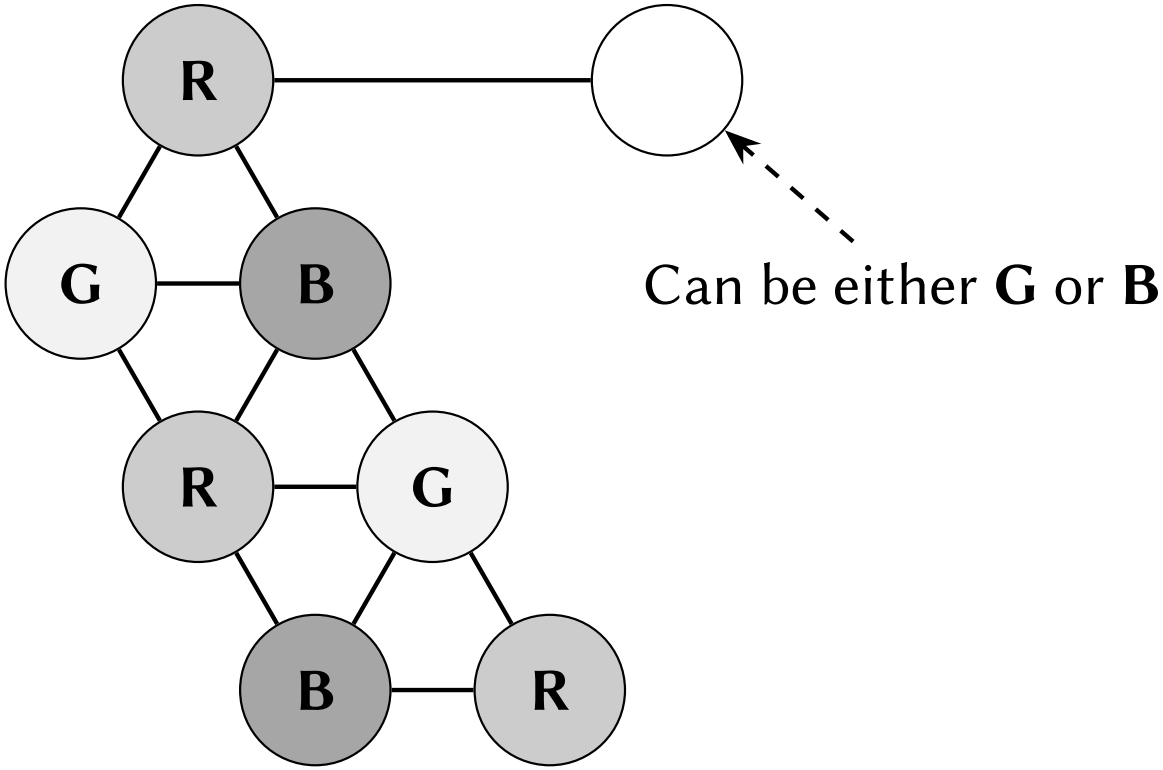
But we can extend the maximally constrained pattern:



We can force our searcher to copy a colour from one node to another.

3COL: A Start (6)

We can also force our searcher to make binary choices. . .



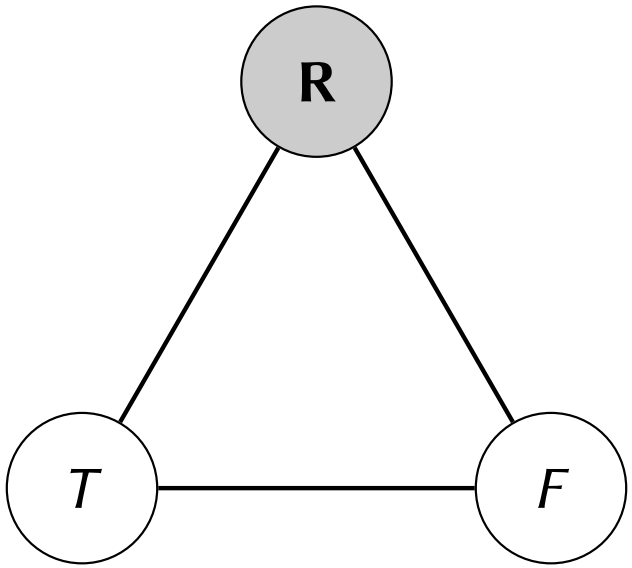
Binary choices are useful – we want to encode truth assignments to binary variables, after all.

3SAT $\leqslant_p$ 3COL

We can actully start encoding 3CNF ϕ now.

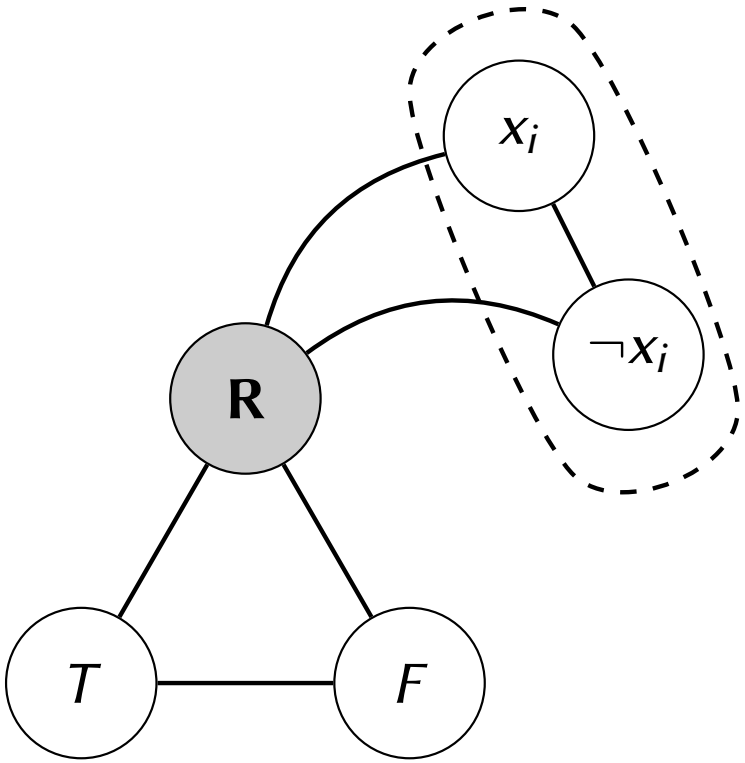
We first rename the colours **G** and **B** to T and F (to reflect truth values).

We then build a triangle to enforce colour choices (a *palette widget*):



3SAT $\leqslant_p$ 3COL (2)

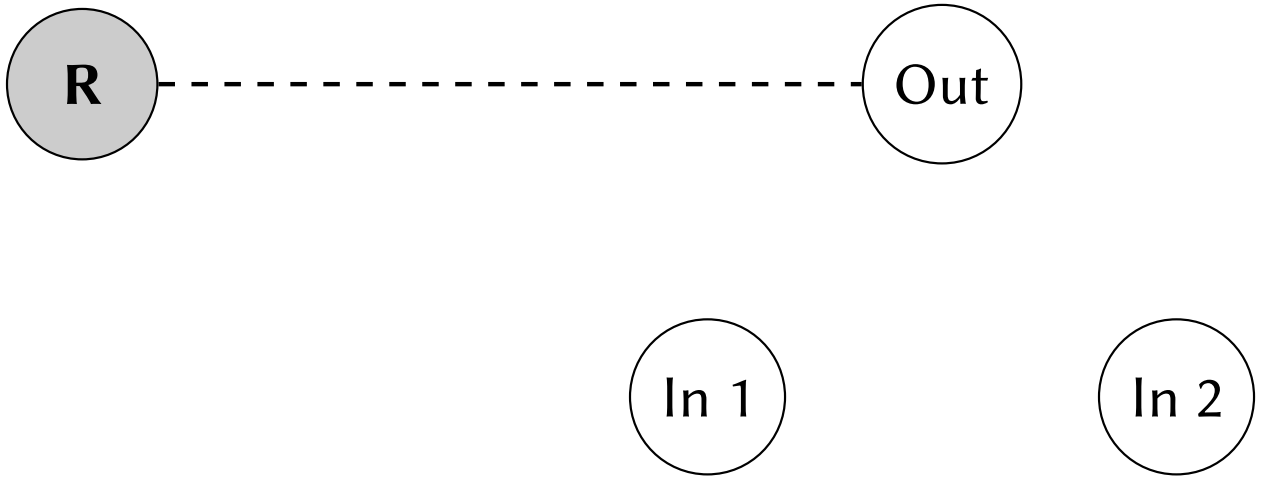
The palette widget lets us force a binary truth assignment to every variable in ϕ:



All we need now are the logic gates.

3SAT $\leqslant_p$ 3COL (3)

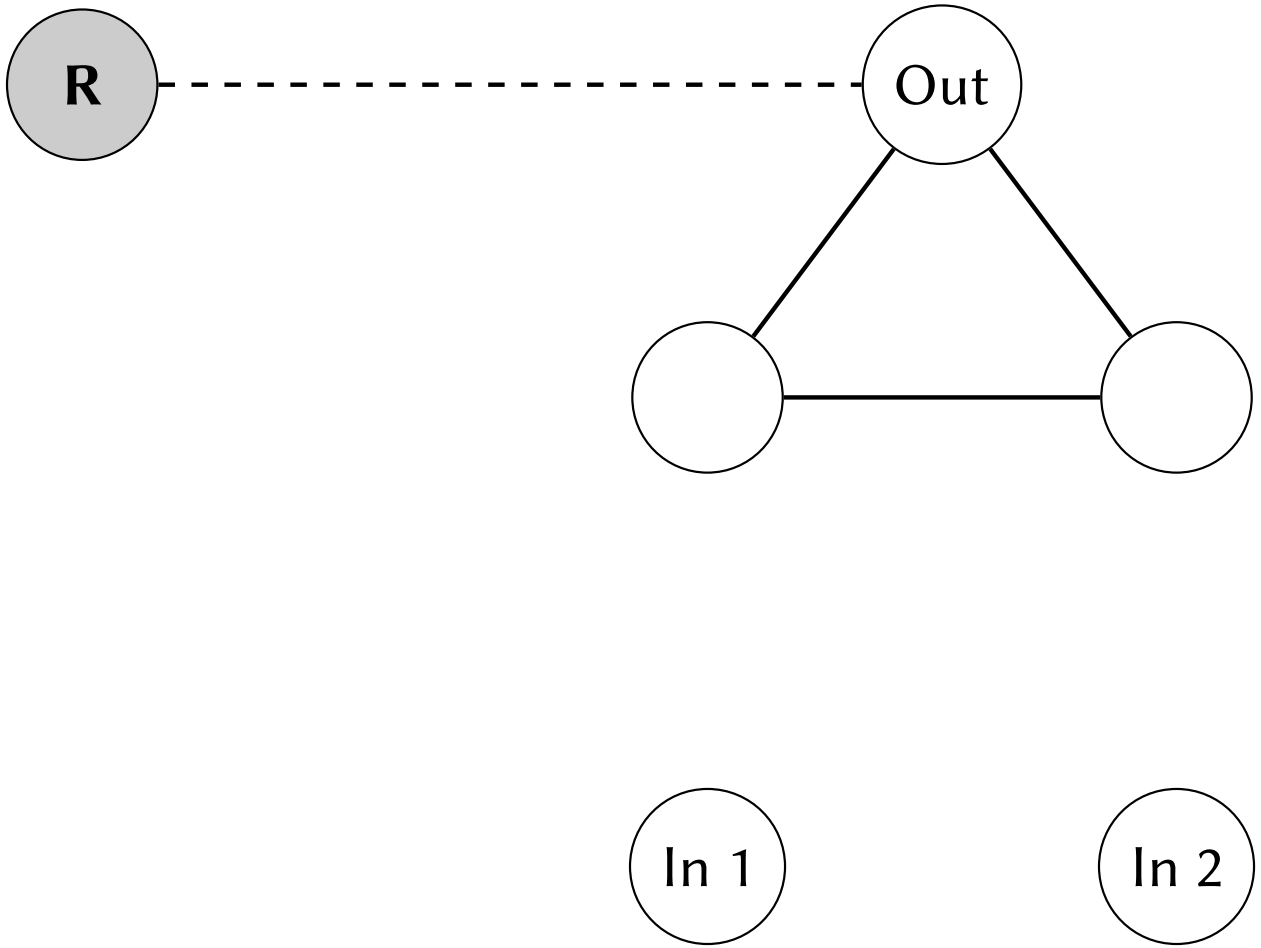
Each clause is an *or*-gate, so let's try building those.
We'll start by building the inputs and outputs:



Notice how we force the output to be *T* or *F*.

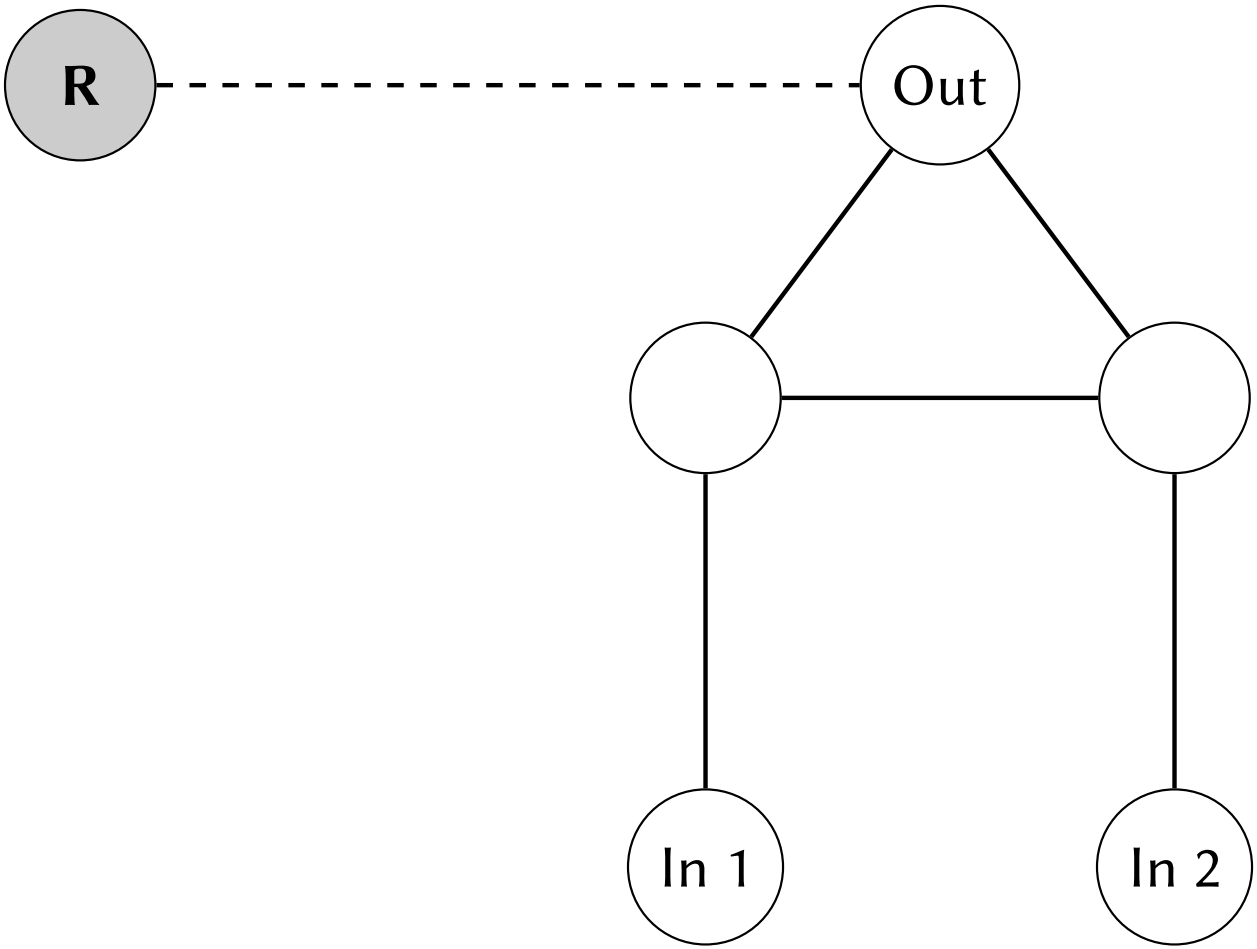
3SAT $\leqslant_p$ 3COL (4)

If we make a triangle out of the output, the other corners must take the other two values:



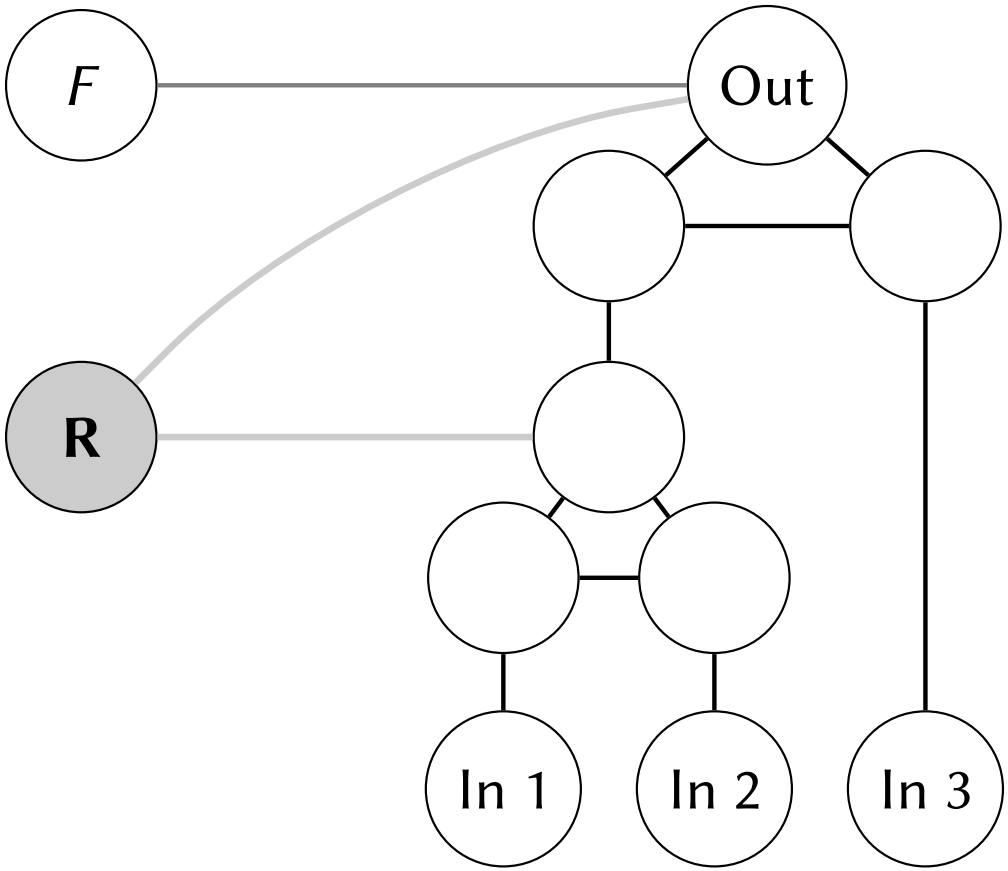
3SAT $\leqslant_p$ 3COL (5)

And if we attch thee corners to the inputs, we get the widget we want:



3SAT $\leqslant_p$ 3COL (6)

We finish by stacking two *or*-gates on top of each other, and then repeating the pattern for every clause:



3SAT $\leqslant_p$ 3COL (7)

To Finish:

- Give an iff proof: ϕ is satisfiable iff G is 3-colourble.

  See the long notes or the Polytime Reductions handout for details.

- Show the reduction is polytime.

  Size of G: if ϕ has ℓ variables and k clauses, then the number of edges and vertices in G is $\mathcal{O}(\ell + k)$. Each vertex and edge can be found using polytime lookups.

  See the Polytime Reductions handout for more details.

Example 2: CUBIC-SUBGRAPH

This is a new language for us:

> **INPUT:** A graph $G = (V, E)$.
>
> **QUESTION:** Does G have a cubic subgraph (i.e., a set $V' \subseteq V$ such that the subgraph of G containing all of the vertices in V' and all of the edges between these vertices will be cubic)?
>
> *A graph G is cubic, or 3-regular, if every vertex has a degree of exactly 3. Also note that we don't accept empty graphs.*

Reduced from: 3COL.

See the Polytime Reductions handout for the proof of membership in $\mathcal{NP}$.

Example 2: CUBIC-SUBGRAPH (2)

If we reduce from 3COL we'll:

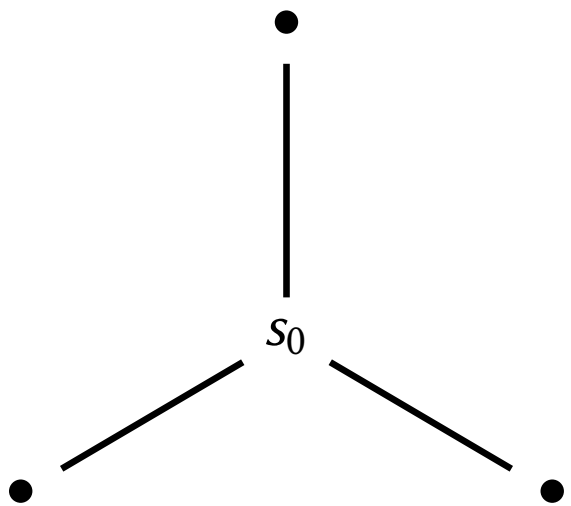
- Take an input 3COL instance $\langle G \rangle$.
- Return an output CUBIC-SUBGRAPH instance $\langle G' \rangle$.
- G' should have a cubic subgraph iff G i 3-colourable.

We'll do this in two steps:

1. We'll use G to build a graph G' with a specified node s_0 such that G' contains a cubic subgraph that contains s_0 iff G has a 3-colouring.
2. We'll argue that every cubic subgraph of G' must contain this node s_0.

Example 2: CUBIC-SUBGRAPH (3)

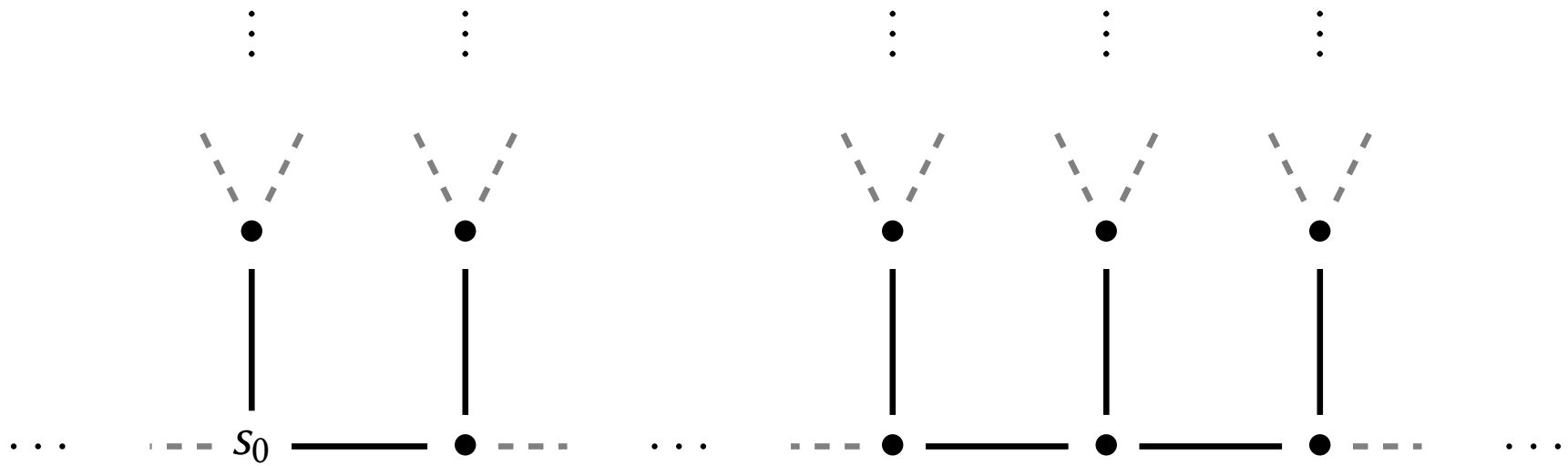
To start off, let's consider a graph G' with a node s_0:



Idea: Any cubic subgraph containing s_0 also contains its neighbours.

Example 2: CUBIC-SUBGRAPH (4)

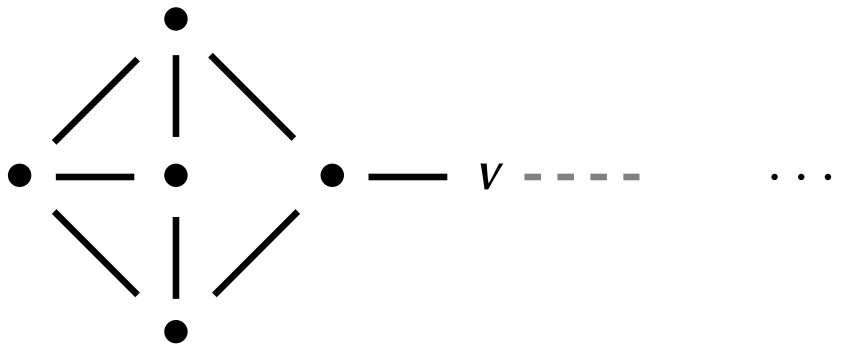
We can extend this pattern:



Any cubic subgraph containing s_0 contains the whole chain.

Example 2: CUBIC-SUBGRAPH (5)

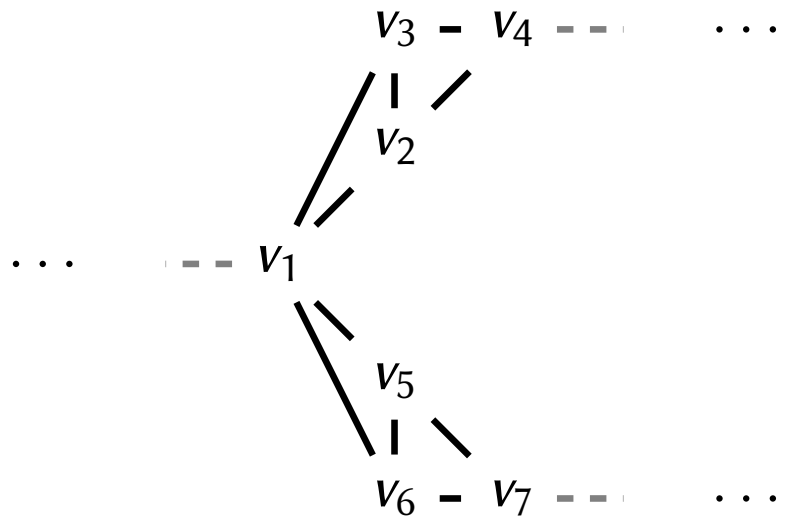
We can cap off the chain at any point we want:



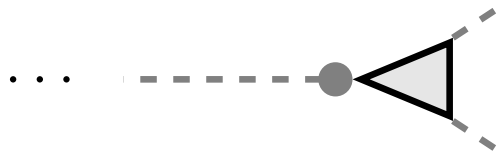
We'll call this widget a *tag*, and denote it as

Example 2: CUBIC-SUBGRAPH (6)

We can also build a choice widget:

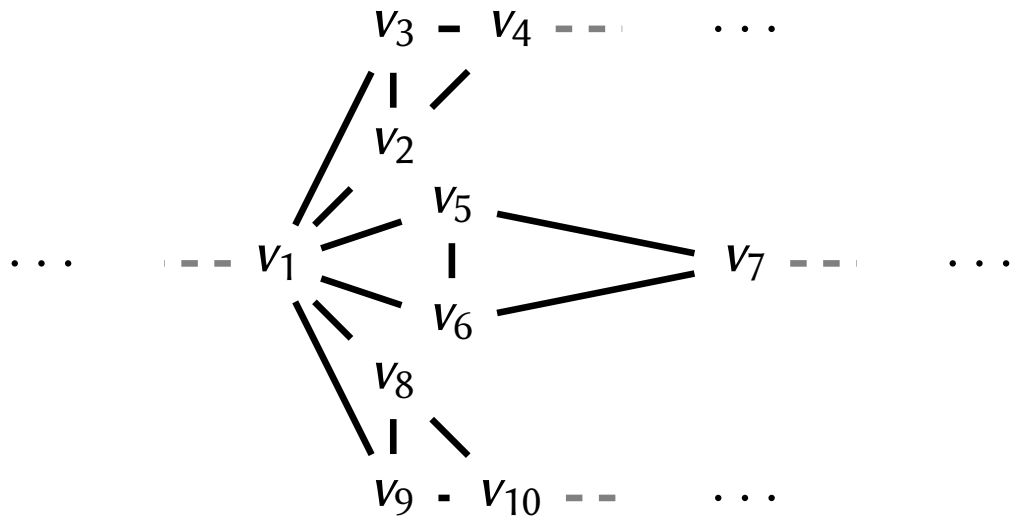


If any node of this widget is in a cubic subgraph, we must include the left branch and <u>one</u> right branch. We'll denote this as:

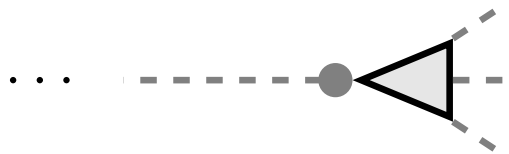


Example 2: CUBIC-SUBGRAPH (7)

We can add as many right branches as we want, though:



Denoted as:

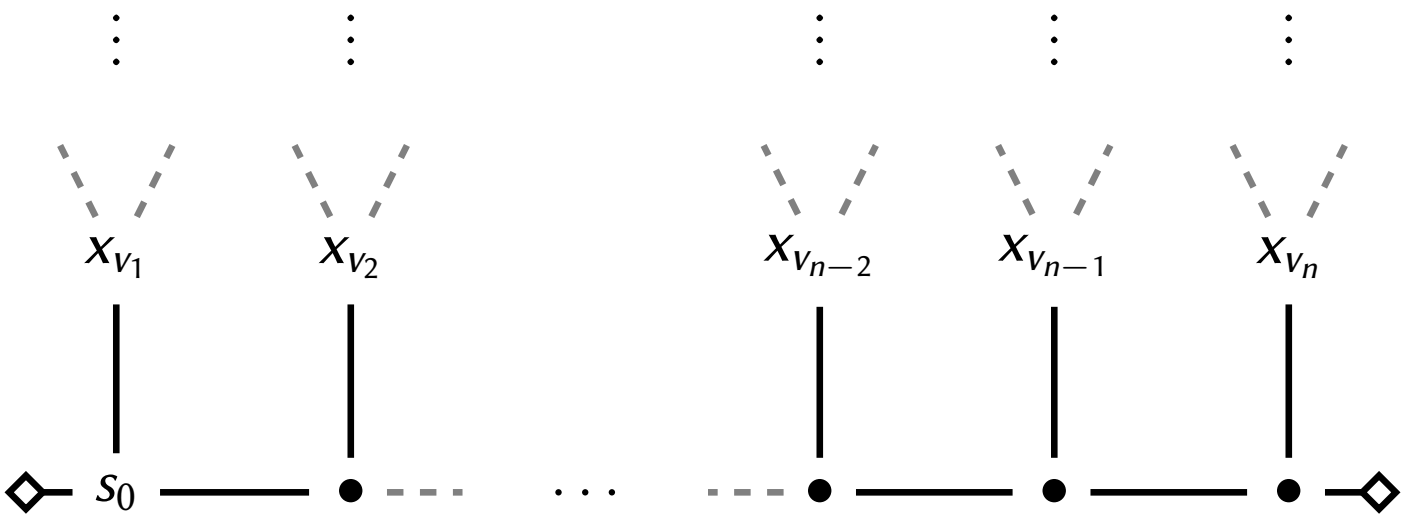


3COL $\leqslant_p$ CUBIC-SUBGRAPH

We can now write the reduction itself.

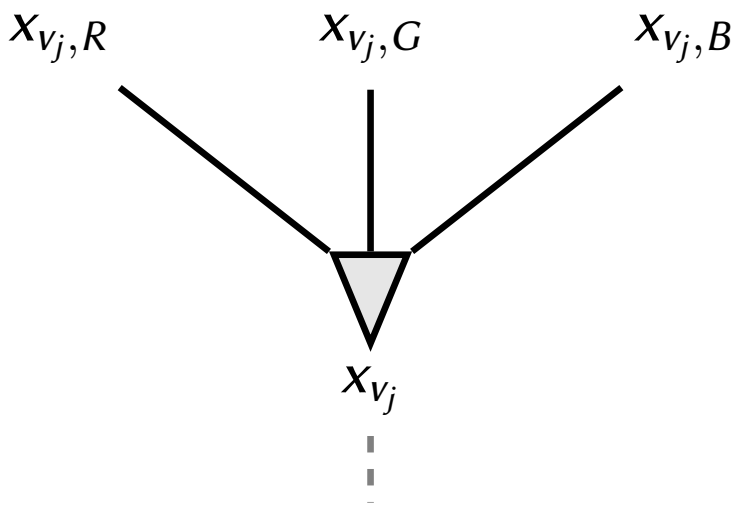
Suppose we have an n-vertex, m-edge graph G.

Start with a chain that has one link per node in G:



3COL $\leqslant_p$ CUBIC-SUBGRAPH (2)

Each link of the chain (one per node in G) will have three choices: one per colour (of the associated G node):

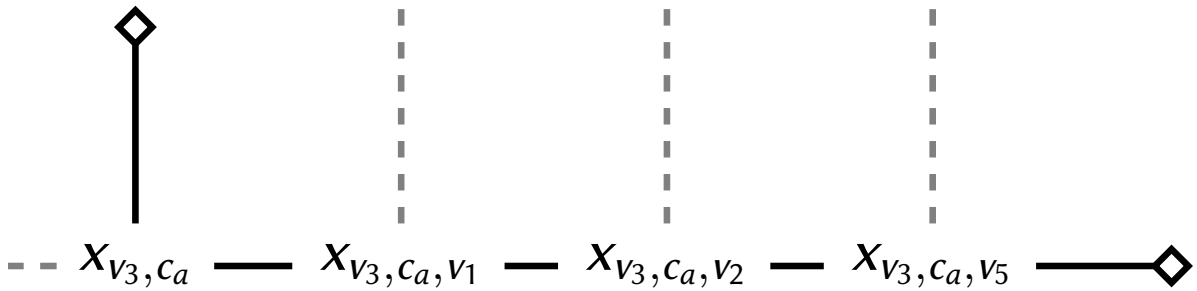


Each choice will attach to a chain.

3COL $\leqslant_p$ CUBIC-SUBGRAPH (3)

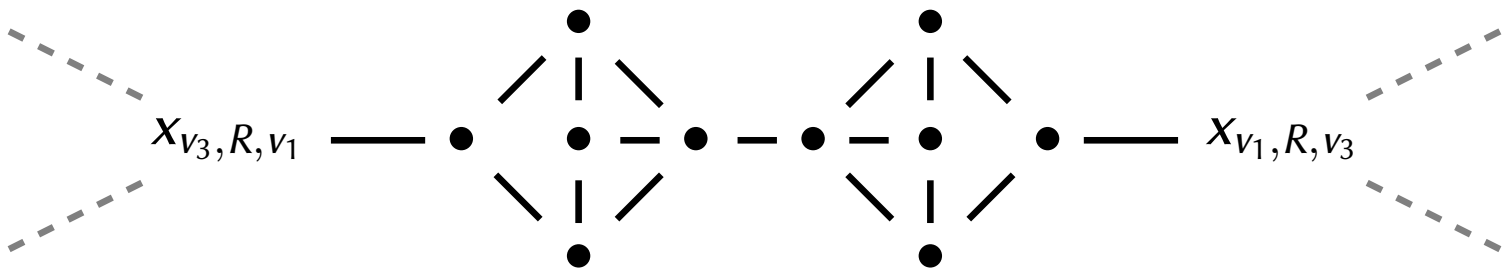
Next, attach each x_{v_j,c_a}, $a \in \{R, G, B\}$, to chains whose length matches the number of neighbours of v_j in G.

For example, if v_3 were adjacent to v_1, v_2, and v_5 then each x_{v_3,c_a} would be connected to chain



3COL $\leqslant_p$ CUBIC-SUBGRAPH (4)

Finlly, we attach the unfinished links to "paired tags":



Any cubic subgraph must contain either the whole left tag, the whole right tag, or none of either.

So we can't assign (in this example) v_1 and v_3 the same colour.

3COL $\leqslant_p$ CUBIC-SUBGRAPH (5)

To Finish:

- Give an iff proof: G is 3-colourable iff G' has a cubic subgraph.

  See the long notes or the Polytime Reductions handout for details.

- Show the reduction is polytime.

  Size of G': if G has n nodes and m edges, then the number of edges and vertices in G is $\mathcal{O}(n+m)$. Each vertex and edge can be found using polytime lookups.

  See the Polytime Reductions handout for more details.

Example 3: CLIQUE

Recall the definition of CLIQUE:

> **Input:** A graph $G$and a number $k \in \mathbb{N}$.
>
> **Question:** Does G have a clique of size k?

We've already showed this is in $\mathcal{NP}$, so let's show that 3SAT $\leqslant_p$ CLIQUE.

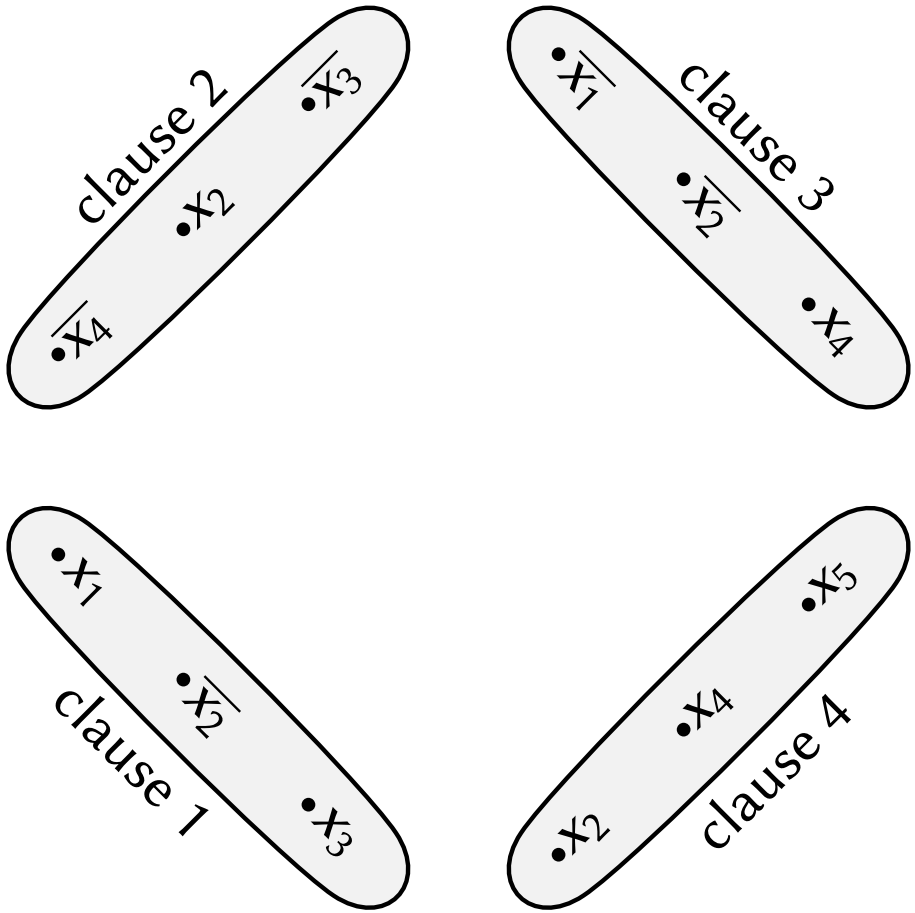
Let's start with the usual input: a 3CNF formula ϕ.

e.g., $(x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_4} \vee x_2 \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_4) \wedge (x_2 \vee x_4 \vee x_5)$

Approach: *Instead of choosing a truth assignment, this time we'll choose one literal per clause to be set to true.*

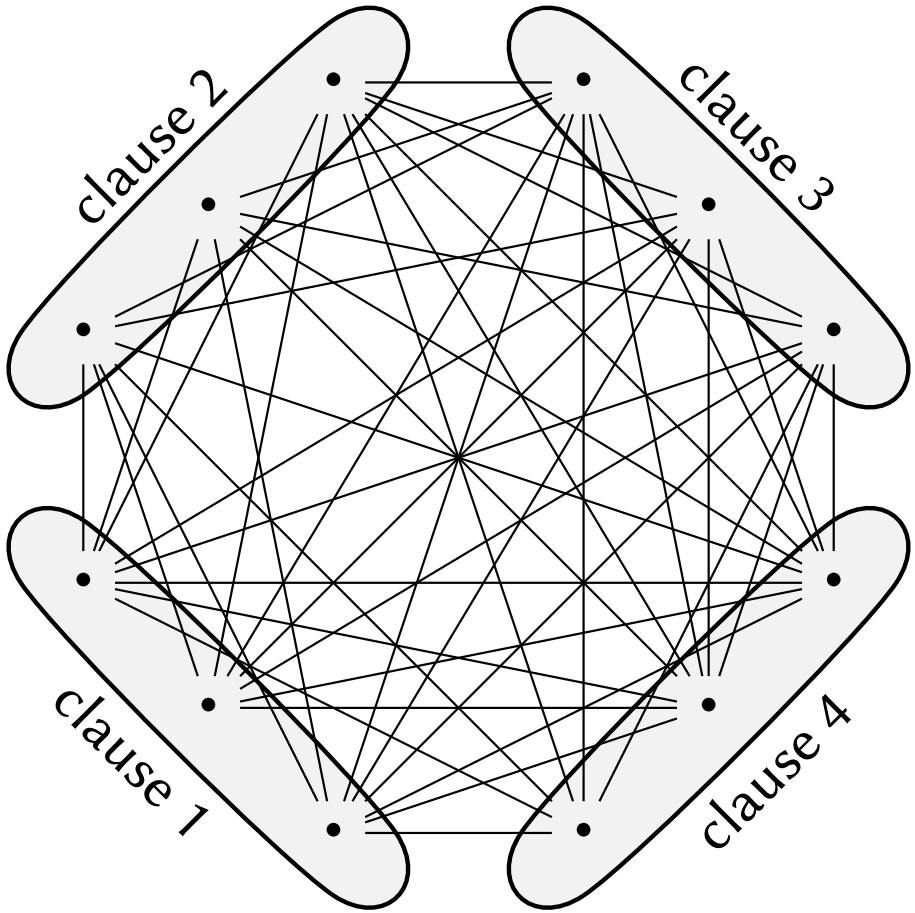
Example 3: CLIQUE (2)

Idea: Build a G so that clause maps to a subset of vertices. Each vertex corresponds to one of the three literals in the clause:



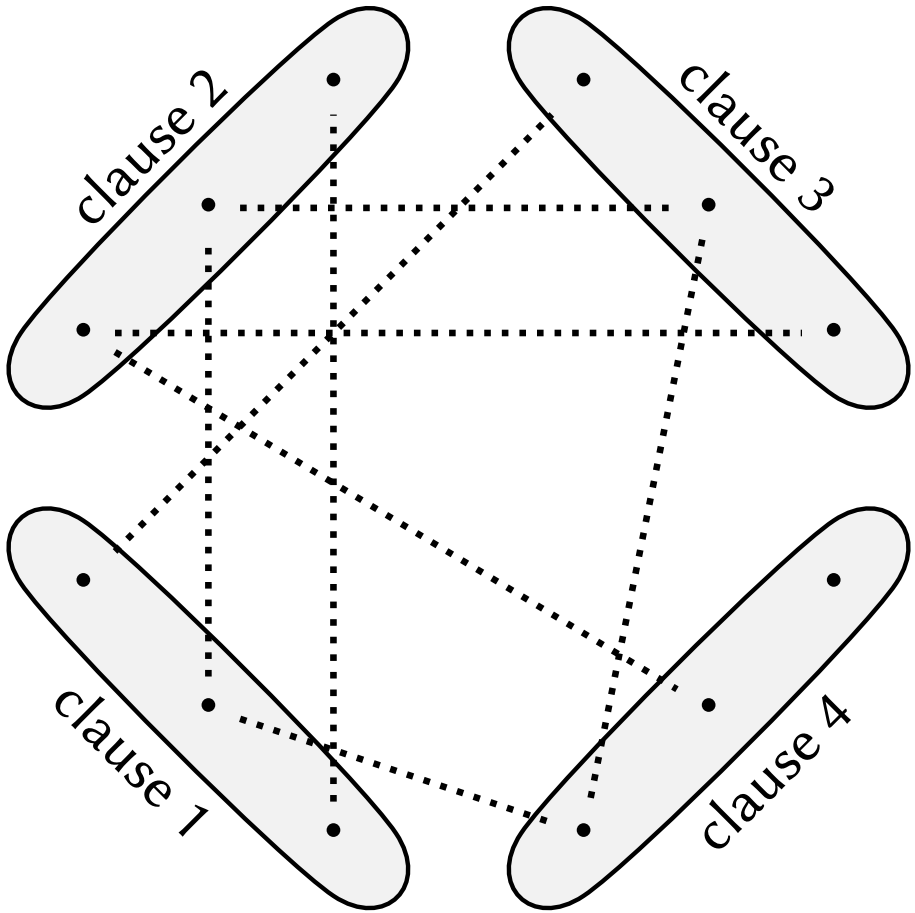
Example 3: CLIQUE (3)

Now, add edges between different subsets whenever those edges could correspond to some consistent truth formula:



It may look better to show the edges we didn't add

The missing edges:



Example 3: CLIQUE (4)

This is the G we return (the first one, not the one with the missing edges). To finish, we simply set k to be the number of clauses in ϕ.

This construction can be made in polynomial time:

> Given a ϕ with a clauses and b variables, the number of vertices in G will be $3a$, and the maximum number of edges will be $9a/2$. We can determine whether each of these edges is in E with at worst $\mathcal{O}(n)$ time, and so the entire reduction is polytime.

Example 3: CLIQUE (5)

G has a k-clique iff $\phi \in$ 3SAT:

($\Leftarrow$): Suppose $\phi \in$ 3SAT:

- It has a satisfying assignment.
- Let S be a set of vertices in G that takes one vertex from each clause group that matches the truth assignment.
- These variables are connected in G.
- $\Rightarrow$ S is a k-clique in G.

($\Rightarrow$): Suppose G has a k-clique S:

- S must have exactly one vertex in each clause group
- Every variable node in S must be chosen consistently.
- We can read the truth values of these variables to get a truth assignment.
- This truth assignment will make ϕ true.
- So this is a satisfying assignment for ϕ.

So 3SAT $\leqslant_p$ CLIQUE, and so CLIQUE is $\mathcal{NP}$-complete.

Example 4: SUBSET-SUM

What do we do if we're looking at a very different sort of problem?

Let's define SUBSET-SUM as:

> **Input:** A multiset S of non-negative integers, an integer t.
>
> **Question:** Is there a subset $S' \subseteq S$ such that $\sum_{x \in S'} = t$?

1. Proof that Subset-Sum is in $\mathcal{NP}$:
    1.
    2.
    3.
    4.
    5.

Example 4: SUBSET-SUM (2)

Remember that to prove 3SAT $\leqslant_p$ SUBSET-SUM, we want to encode a 3SAT instance into a SUBSET-SUM instance.

Given a 3SAT instance ϕ, we'll try to follow three basic steps:

- Find a way to code SUBSET-SUM to reflect the truth assignments for ϕ.
- Extend the resulting instance to reflect the resulting truth values in the clauses.
- Add any fudge factors we need to balance everything out.

Example 4: SUBSET-SUM (3)

So, given an input 3CNF ϕ, we'll write the variables of ϕ as $x_1, \ldots, x_\ell$ and its clauses as $c_1, \ldots, c_k$.

Idea: Let's start by setting up a table like so:

	1	2	3	...	ℓ	
x_1	1	0	0	...	0	$\langle$ something $\rangle$
$\overline{x_1}$	1	0	0	...	0	$\langle$ something $\rangle$
x_2	0	1	0	...	0	$\langle$ something $\rangle$
$\overline{x_2}$	0	1	0	...	0	$\langle$ something $\rangle$
x_3	0	0	1	...	0	$\langle$ something $\rangle$
$\overline{x_3}$	0	0	1	...	0	$\langle$ something $\rangle$
$\vdots$				$\vdots$		$\vdots$
x_ℓ	0	0	0	...	1	$\langle$ something $\rangle$
$\overline{x_\ell}$	0	0	0	...	1	$\langle$ something $\rangle$
	1	1	1	...	1	$\langle$ something $\rangle$

So two rows per variable.

Example 4: SUBSET-SUM (4)

Suppose that we want to choose a set of rows whose columns add up to the numbers on the bottom. Then, for any x_i, we would have to choose either the row headed by x_i or the one headed by $\overline{x_i}$. We wouldn't be able to choose both.

This is a bit like forcing every boolean variable x_i to be either true or false - the rows we'd choose above correspond to truth assignments!

We'll use the $\langle$ something $\rangle$s on the right to encode the clauses.

Example 4: SUBSET-SUM (5)

Suppose that c_1 is $(x_1 \vee \overline{x_2} \vee x_3)$. We can encode this in the first column on the right like so:

	1	2	3	...	ℓ	c_1	...
x_1	1	0	0	...	0	1	...
$\overline{x_1}$	1	0	0	...	0	0	...
x_2	0	1	0	...	0	0	...
$\overline{x_2}$	0	1	0	...	0	1	...
x_3	0	0	1	...	0	1	...
$\overline{x_3}$	0	0	1	...	0	0	...
$\vdots$				$\vdots$			$\vdots$
x_ℓ	0	0	0	...	1	0	...
$\overline{x_\ell}$	0	0	0	...	1	0	...
	1	1	1	...	1	?	...

That is, we've added a 1 in this column to every row corresponding to a literal in c_1.

Example 4: SUBSET-SUM (5)

We can repeat the same process for every clause:

	1	2	3	...	ℓ	c_1	c_2	...	c_k
x_1	1	0	0	...	0	1	0	...	0
$\overline{x_1}$	1	0	0	...	0	0	0	...	0
x_2	0	1	0	...	0	0	1	...	0
$\overline{x_2}$	0	1	0	...	0	1	0	...	0
x_3	0	0	1	...	0	1	0	...	0
$\overline{x_3}$	0	0	1	...	0	0	1	...	1
$\vdots$				$\vdots$				$\vdots$	
x_ℓ	0	0	0	...	1	0	0	...	1
$\overline{x_\ell}$	0	0	0	...	1	0	0	...	0
	1	1	1	...	1	?	?	...	?

Example 4: SUBSET-SUM (6)

Any satisfying truth assignment to ϕ gives us something between 1 and 3 in the second set of columns. Any non-satisfying assignment will set at least one clause from the second column to 0.

If we add a couple of dummy variables to every clause column, we can make them add up to 3 when there's a satisfying assignment:

	1	2	3	...	ℓ	c_1	c_2	...	c_k
$\vdots$				$\vdots$				$\vdots$	
d_1						1	0	...	0
d'_1						1	0	...	0
d_2						0	1	...	0
d'_2						0	1	...	0
$\vdots$								$\vdots$	
d_k						0	0	...	1
d'_k						0	0	...	1
t	1	1	1	...	1	3	3	...	3

Example 4: SUBSET-SUM (7)

Putting it all together:
To turn this into a SUBSET-SUM instance, just note that the numbers in the columns can never add up to more than 3 – so we can treat the rows as numbers in base 10 and never have to worry about one digit affecting another. So our S will just be the non-bottom rows in the table, and t will be the bottom row.

We can see that this construction will take polynomial time:

> The total table size is $(\ell + k)^2$, which is clearly polynomial in the size of ϕ. Each entry in the table is a 0, a 1, or a 3, and each value can be found with at worst a polynomial time lookup. The string value of t, $\langle t \rangle$ is $1^\ell 3^k$, which can also be found in polynomial time. So all told, this construction takes polynomial time.

And we've already given an overview of our justification as to why it works. So 3SAT $\leqslant_p$ SUBSET-SUM, and so SUBSET-SUM is $\mathcal{NP}$-complete.

Example 6: INDEPENDENT-SET (IS)

Definition of INDEPENDENT-SET *(IS)*:

Instance: A graph $G = (V, E)$ and a number $k \in \mathbb{N}$.

Question: Does G contain an independent set of size k or more (i.e., a set I of k vertices in V such that, for every pair $\{u, v\} \in I$, G does not have an edge $\{u, v\} \in E$?

Certificate: The independent set I.

Verifier: "On $\langle G = (V, E), I \rangle$:"

1. Check that I has at most $\min(k, |V|)$ elements.
2. For all pairs $\{u, v\}$, $u, v \in I$:
3. Check that $\{u, v\} \notin E$.
4. If all of these checks pass, *accept*, otherwise *reject*.

INDEPENDENT-SET is $\mathcal{NP}$-Complete

Reduction from CLIQUE**:**

Suppose we are given a CLIQUE instance $\langle G = (V, E), k \rangle$ where G has n vertices and m edges.

We return the pair $\langle G' = (V, \overline{E}), k \rangle$, where
$\overline{E} = \{\{u, v\} \mid u \neq v \in V \wedge \{u, v\} \notin E\}$. *(Is this polytime?)*

Proof that $\langle G, k \rangle \in$ CLIQUE **iff** $\langle G', k \rangle \in$ INDEPENDENT-SET**:**
Suppose G has a size-k clique C:

- Then for any $u \neq v \in C$, $\{u, v\} \in E$ in G.
- Then for any $u \neq v \in C$, $\{u, v\} \notin \overline{E}$ in G'.

  $\Rightarrow$ C is a size-k independent set in G'.

Suppose G' has a size-k independent set I:

- Then for any $u \neq v \in I$, $\{u, v\} \notin \overline{E}$ in G'.
- Then for any $u \neq v \in I$, $\{u, v\} \in E$ in G.

  $\Rightarrow$ I is a size-k clique in G.

Example 6: VERTEX-COVER (VC)

Definition of VERTEX-COVER *(VC)*:

Instance: A graph $G = (V, E)$ and a number $k \in \mathbb{N}$.

Question: Does G contain a vertex cover of size k or less (i.e., a set V' of k vertices in V such that, for every edge $\{u, v\} \in E$, at least one of u or v is in V')?

Certificate: The vertex cover V'.

Verifier: "On $\langle G = (V, E), V' \rangle$:"

1. Check that V' has at most $\min(k, |V|)$ elements.
2. For all edges $\{u, v\} \in E$:
3. Check that at least one of u or v is in V'.
4. If all of these checks pass, *accept*, otherwise *reject*.

VERTEX-COVER is $\mathcal{NP}$-Complete

Reduction from INDEPENDENT-SET**:**

Suppose we are given an INDEPENDENT-SET instance $\langle G = (V, E), k \rangle$ where G has n vertices and m edges.

We return the pair $\langle G, n - k \rangle$. *(Is this polytime?)*

Proof that $\langle G, k \rangle \in$ INDEPENDENT-SET **iff** $\langle G, n - k \rangle \in$ VERTEX-COVER**:**
Suppose G has a size-k independent set I:

- Then for any $u \neq v \in I$, $\{u, v\} \notin E$
- Then for any $\{u, v\} \in E$, at least one of u or v is in $V \setminus I$.

  $\Rightarrow V \setminus I$ is a size-$(n - k)$ vertex cover in G.

Suppose G has a size-$(n - k)$ vertex cover V':

- Then for any $\{u, v\} \in E$, at least one of u or v is in V'.
- Then for any $u \neq v \in V \setminus V'$, $\{u, v\} \notin E$

  $\Rightarrow V \setminus V'$ is a size-$(n - (n - k)) = k$ independent set in G.

Example 7: PARTITION-INTO-TRIANGLES

Definition of PARTITION-INTO-TRIANGLES:

Instance: A graph $G = (V, E)$.

Question: Can G be partitioned into disjoint triangles (i.e., can the vertices in V be split into pairwise disjoint sets of three such that each set is a trinagle in G)?

Certificate: The partitioning P
(we can treat this as a collection of triples of vertices in V).

Verifier: "On $\langle G = (V, E), P \rangle$:"

1. . Check that P lists each $v \in V$ exactly once in its triples, and that there are no extra vertices in P).
2. For all triples $\{a, b, c\} \in P$:
3. Check that $\{a, b\}$, $\{a, c\}$, and $\{b, c\}$ are all edges in E.
4. If all of these checks pass, *accept*, otherwise *reject*.

Check: is this polytime?

Why is PARTITION-INTO-TRIANGLES $\mathcal{NP}$-complete?

We reduce from 3SAT.

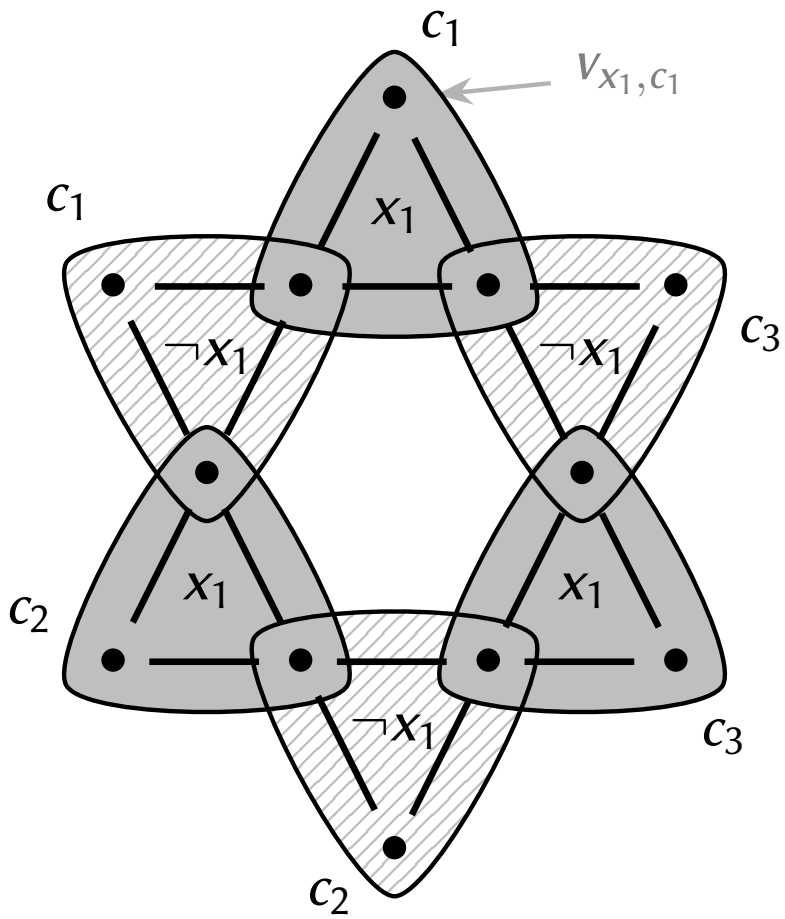
Suppose we are given an input 3CNF ϕ.

$$\text{e.g., } \phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

- How can we program variable widgets into a graph G?
- How can we program the clauses?
- Will we need to do a bit extra to finish the job?

A Variable Widget

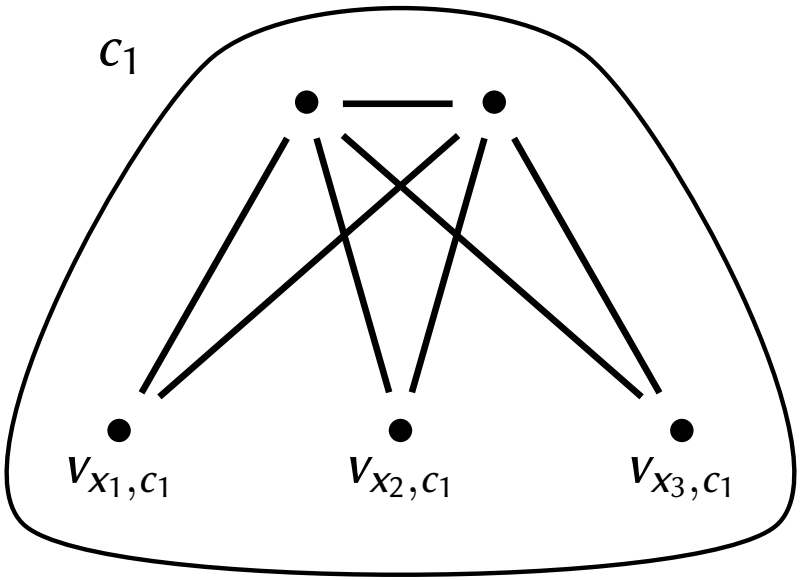
Here's a widget for x_1. Notice that it has two "leaves" for every clause C_j.



We'll connect the points of the "leaves" to the different clauses.

A Clause Widget

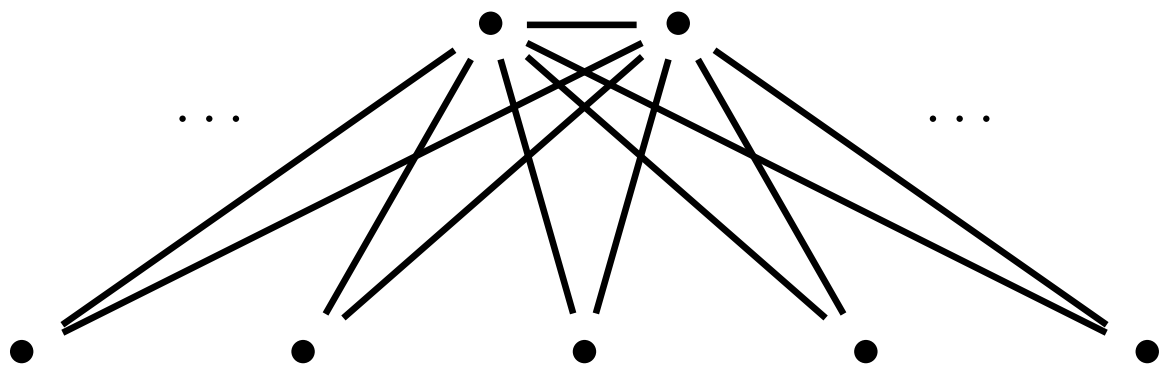
We can just attach the base of a triangle to each leaf corresponding to its literal:



We're almost there, but we've got a bunch of vertices left over. . .

A Garbage Collection Widget

It's basically like a large clause widget, but connected to every leaf in every variable widget:



We put in just enough of these to cover all the vertices (use a counting argument to see how many).

Is this reduction polytime?

Proof that $\phi \in$ 3SAT iff $\langle G \rangle \in$PARTITION-INTO-TRIANGLES

Suppose that $\langle \phi \rangle \in$ 3SAT, with n clauses and k variables:

- Then there exists some truth assignment that assigns at least one literal per clause to true.
- If we use that truth assignment to choose an assignment to triangles for the variable widgets, these literals will be free to be assigned with the n clause widgets.
- This will assign all of the inner nodes from the variable widgets to triangles, and will also assign all of the nodes from clause widgets to triangles. But it will leave $(k-1)n$ of the outer nodes from the variable widgets to be assigned.
- By assigning these nodes to the extra widgets from end of the construction.
- This will give us a decomposition of G into triangles.

  $\Rightarrow \langle G \rangle \in$ PARTITION-INTO-TRIANGLES.

Proof that $\phi \in$3SAT iff $\langle G \rangle \in$PARTITION-INTO-TRIANGLES (2)

Suppose that $\langle G \rangle \in$ PARTITION-INTO-TRIANGLES:

- Then There is some way of breaking G into disjoint triangles.
- In this partitioning, each of the clause widgets must contribute to exactly one triangle.
- The extra vertex in each of these triangles must correspond to some literal on the outside of the variable widgets.
- Since the variable widget enforces consistency in the variable assignments, the literals chosen by the clause widgets are consistent.
- $\Rightarrow$ These literal assignments can be extended into a consistent truth assignment.
- Since this truth assignment sets every clause to true, it is a satisfying assignment for ϕ.

  $\Rightarrow \langle \phi \rangle \in$ 3SAT.

3DM is $\mathcal{NP}$-Complete:

Recall the definition of 3DM:

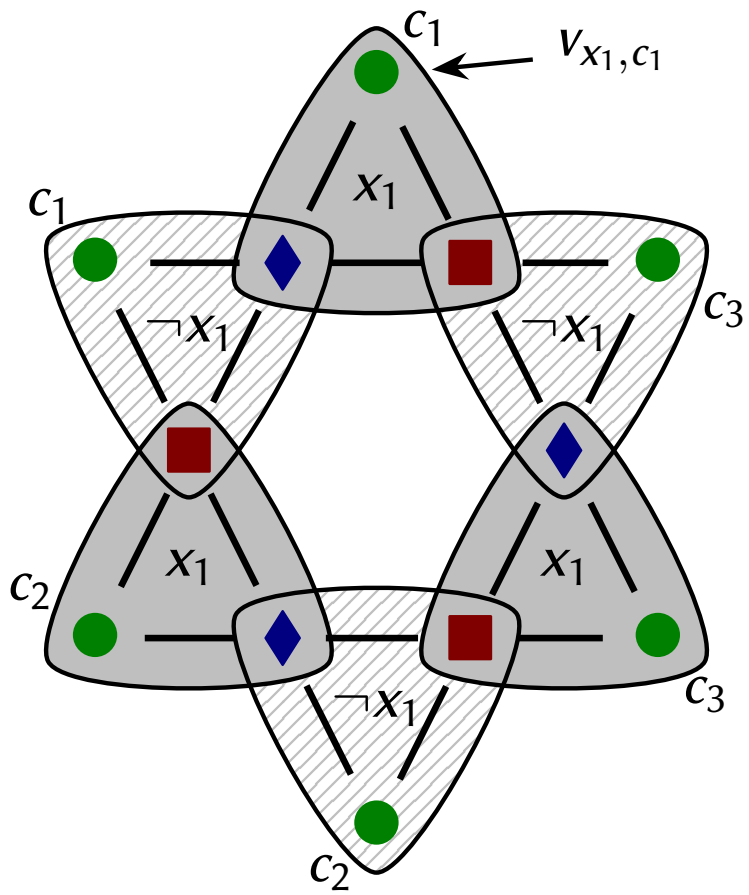
> **Instance:** Three sets A, B, and C, where $|A| = |B| = |C|$, and a set $T \subseteq A \times B \times C$.
>
> **Question:** Is there a subset $M \subseteq T$ such that:
>
> i) $|M| = |A|$,
>
> ii) For any distinct (a, b, c) and $(a', b', c') \in M$, $a \neq a'$, $b \neq b'$, and $c \neq c'$?

Idea: Re-use the reduction from 3SAT to PARTITION-INTO-TRIANGLES.

- We actually build the same graph G.
- We 3-colour G (3-colouring is usually hard, but not in this case).
- The sets A, B, and C are the *green*, *red*, and *blue* vertices.
- The set T is the set of triangles in G.

Colouring the Variable Widget

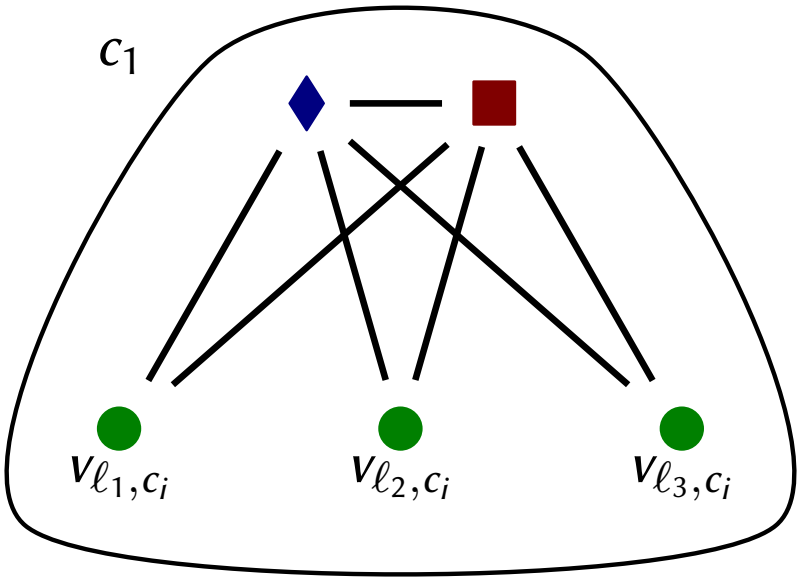
Here is one colouring for the variable widget:



Note: A = set of green circles, B = set of red squares, and C = set of blue diamonds.

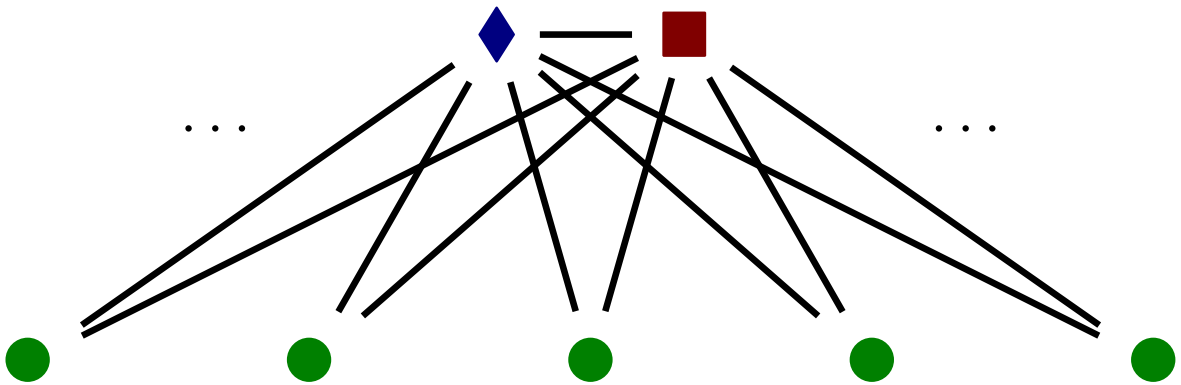
Colouring the Clause Widget

And we can extend this colouring to the clause widget:



Colouring the Garbage Collection Widget

And we can finish the colouring with the garbage collection widget:



If ϕ has n clauses and k vertices:

$$\begin{aligned} & \text{Number of green circles} && (|A|) \\ = \ & \text{number of red squares} && (|B|) \\ = \ & \text{number of blue diamonds} && (|C|) \\ = \ & 2nk. \end{aligned}$$

For the iff proof: See the PARTITION-INTO-TRIANGLES *reduction.*